Sparse Goal Edits and Variable-Duration Options in World-Model Hierarchical RL

George Vasilache

Working draft — July 29, 2026

Abstract

We study whether the manager of a Director-style hierarchical agent built on DreamerV3 can act more sparsely — editing only part of a persistent goal per decision and choosing how long each goal is held — without losing control performance. On dense-reward cartpole the combined recipe reaches returns of ∼840 while editing fewer goal dimensions per step than the fixed-schedule baseline, and outperforms each of its components in isolation; on cheetah, replacing fixed penalty weights with Lagrangian constraint controllers roughly doubles the return. On sparse-reward tasks (hopper hop, acrobot swingup) every restricted variant fails to ever encounter reward, while the unrestricted baseline hierarchy learns both. Controlled comparisons isolate two candidate mechanisms — short goal durations and an unobservable, random per-goal horizon on the worker side — and we extend the agent with timed subgoals (a countdown-conditioned worker, following Gürtler et al. 2021) to test the latter. A factorial experiment across duration target, structural code regularization, and countdown conditioning is in progress; preliminary signal (single seed, 50–83% of budget) shows the countdown substantially rescues a collapsing dense-task variable-duration run but does not move either sparse task, and that masked edits alone (fixed schedule, no variable duration) are *independently* sufficient to remove hopper's reward too — so the two restrictions are each, on their own, enough to break sparse-task learning.

1 The base agent

The base agent is a reimplementation of Director (Hafner et al. 2022) on DreamerV3. All components below are standard; the paper is self-contained given this section.

World model. A recurrent state-space model (RSSM) is trained on replayed experience to predict observations, rewards, and episode continuation. It provides latent states $s_t = (\text{deter}_t, \text{stoch}_t)$, and both policies are trained entirely on *imagined* rollouts of this model (horizon H=16), not on real transitions.

Goal autoencoder. Goals are latent states compressed through a discrete autoencoder: an encoder maps a target latent to a code $z \in \{1, \dots, C\}^L$ of L categorical blocks (L=8, C=8 here), a decoder maps codes back to latent goals. The manager acts in code space; the worker receives the decoded latent goal g.

Manager. Every K steps (K=8 in Director) the manager policy $\pi^{\text{mgr}}(z \mid s)$ emits a goal code. It is trained by REINFORCE on imagined rollouts to maximize task (extrinsic) reward plus a novelty (exploration) reward with weight 0.1, each with its own critic. The per-decision reward is the *mean* of the per-step rewards over the goal's hold window, so the manager's return is invariant to how finely time is partitioned. An entropy constraint holds the manager's normalized policy entropy near a target of 0.5.

Worker. The worker policy $\pi^{\text{wkr}}(a \mid s, g)$ receives the state features and the decoded goal, and is trained on a dense per-step reward: the (max-)cosine similarity between the current latent and the goal. Its λ-return is cut at goal switches, so its value function $V(s, g)$ bootstraps only within the current goal's window. The worker *never* observes task reward. This division — manager selects goals for task reward, worker reaches goals — is what the rest of the paper perturbs.

2 Two manager restrictions

We ask whether manager decisions can be made *sparser* along two axes, with the long-term aims of interpretability (attributable, local goal edits), a controllable temporal-abstraction knob, and less non-stationarity in the worker's objective (most of the goal persists across decisions).

Masked goal edits. The manager maintains a persistent goal code Z_t and at each decision outputs a candidate code z_t together with a binary mask $m_t \in \{0, 1\}^L$ over the L blocks, updating

$$Z_t \;=\; m_t \odot z_t \;+\; (1 - m_t) \odot Z_{t-1}, \tag{1}$$

so only the selected blocks change. The mask is a learned policy output; its sparsity is controlled by constraints (Sec. 3), not hard-coded.

Single-head (joint) edits. The two-head form above has a credit-assignment flaw: the content head proposes a value $z_t[i]$ for *every* block on every decision, and REINFORCE reinforces its log-probability regardless of whether the mask $m_t[i]$ discarded it. At edit rate $\bar{m}$ a fraction $1-\bar{m}$ of the content gradient is thus applied to proposals that never influenced the trajectory — causally disconnected noise (a driver of the unstable manager value we observe in the two-head runs). We remove this by construction: merge the content and mask heads into a *single* per-block categorical over $C{+}1$ classes, where class 0 = *abstain* (keep $Z_{t-1}[i]$) and classes $1..C$ = overwrite with content class c. Then $Z_t[i]$ is a function of one sample with one log-probability per block; when the manager abstains, abstaining *is* the entire recorded action for that block, so there is no discarded content event to miscredit. The edit rate $\bar{m} = 1 - \Pr[\text{abstain}]$ is either left free (shaped only by task-return REINFORCE) or ratcheted: Pr[abstain] is driven from 0 (full Director-style goal edits early) to 0.7 over ∼1M steps by the same rate controller (Sec. 3) acting on the non-abstain probability.

Variable durations. Alongside the goal, the manager outputs a hold duration $d_t \in \{1, \ldots, 16\}$ from a categorical head; the next decision occurs after exactly d_t steps (the worker is never interrupted early).

The combined cost measure is the average number of goal blocks edited per environment step,

$$\text{blk/step} \;=\; \frac{\mathbb{E}[\|m\|_1]}{\mathbb{E}[d]} \;=\; \frac{\bar{m}\,L}{\bar{K}}, \tag{2}$$

with $\bar{m}$ the mean edit fraction and $\bar{K}$ the realized mean duration. Director corresponds to $\bar{m}$=1, $\bar{K}$=8: blk/step = 1.

3 Learning machinery

3.1 Constraint controllers

Direct fixed penalties on edit fraction or duration proved brittle and task-specific. The current recipe controls both with dual-ascent Lagrangian controllers of a common form: for a statistic x with target τ, a multiplier λ on the term λx in the manager loss is updated

$$\lambda \leftarrow \text{clip}\big(\lambda \cdot \exp(\eta\,(x-\tau)),\ \lambda_{\min},\ \lambda_{\max}\big). \tag{3}$$

Three controllers are active in the combined recipe: (i) **edit rate**: mean mask probability → 0.3; (ii) **per-block mask entropy**: → 0.5, preventing collapse onto always/never-edited blocks; (iii) **duration**: the switch-weighted deviation $|\mathbb{E}[d] - \tau_d| \to$ tolerance 0.1 (with $\tau_d \in \{4, 8\}$); regulating the deviation keeps the controller symmetric, and it relaxes to $\lambda \to 0$ once within tolerance.

3.2 Structural code consistency

A structural loss with weight w_s (200 unless stated) ties code-space distances to latent-state distances so that small code edits correspond to nearby goals:

$$\mathcal{L}_{\text{struct}} = \big\|\text{dist}(z_i, z_j) - \text{dist}(s_i, s_j)\big\|^2, \qquad \text{realized corr.} \approx 0.92\text{–}0.98. \tag{4}$$

This acts on the goal autoencoder independently of masking. It has two faces: it *stabilizes* what codes mean while the autoencoder trains on an expanding state distribution, and it *localizes* proposals (small code moves = small goal moves), which may suppress exploration on sparse tasks. Separating these two effects is one goal of the current experiments. A first adaptive variant regulated w_s (Eq. 3) toward a small, two-sided setpoint (0.01) on the raw structural loss; diagnosing why it underperformed a fixed w_s=200 on cheetah (e198 vs. e174, single seed) showed the raw loss clears that setpoint almost immediately, collapsing the multiplier from its init of 200 to ∼20–30 within the first 7% of training and leaving the diagnostic correlation (Eq. 4) 3–6 points below the fixed-weight run's for the remainder. Retargeting onto the correlation directly (0.97) was considered but rejected after checking the fixed-weight runs' own trajectories: at BIG scale the correlation never reaches 0.97 even under w_s=200 held constant for all 4M steps (e171 plateaus at 0.90–0.92, e175 at 0.94), so a 0.97 floor would simply ratchet the multiplier to its ceiling and pin it there for the whole run. The setpoint instead stays on the raw loss, lowered to 0.005 (below every fixed-weight run's floor at either scale) to 0.005 (below every fixed-weight run's floor at either scale), keeping the symmetric, two-sided form of Eq. (3) (grows above target, shrinks below). The controller implementation also gained

an optional one-sided mode — grows on violation but never relaxes once the target clears — motivated by the same fixed-weight runs showing the loss can drift back above a setpoint later in training with no relaxation mechanism at play at all (e171: 0.0068 at 445k steps to 0.0183 at 3.1M), which would let a symmetric controller shrink early and then lag on re-detecting the later violation; it is available but not the default while the standard dual-ascent form is evaluated first. Results under this variant are pending.

3.3 Timed subgoals: countdown-conditioned worker

In the base agent the worker observes only (s, g): it cannot see how long it has to reach the goal, and under variable durations its value target has a *random, unobservable* horizon (1–16 steps, mean τ_d) — variance injected directly into its critic, on top of shorter reach-times at τ_d=4. Empirically, worker goal-reward is lowest exactly where variable-duration runs collapse. Following HiTS (Gürtler et al., NeurIPS 2021), which shows that a lower level pursuing *timed* subgoals $(g, \Delta t)$ restores a stationary learning problem, we append the **countdown** — steps left on the current goal including the current one, normalized to $[-1, 1]$ by the duration budget — to the worker's policy and value inputs (`worker_timed_goals`). Our variable-duration manager already implements the manager half of timed subgoals (durations fixed in advance, no early interruption); this supplies the missing worker half. The HiTS machinery that relies on off-policy replay relabeling (hindsight action relabeling, HER with time relabeling, deadline-only sparse rewards) does not transfer to imagination training and is not used; the worker keeps its dense cosine reward, so the experiment isolates *horizon observability*. The formal definition of the countdown and its place in the worker's losses is given in Sec. 3.4.

3.4 The complete objective

All losses are combined as a scale-weighted sum over independent keys, $\mathcal{L} = \sum_k s_k \mathcal{L}_k$; the structural invariant carried through this project (Sec. 6, finding 2) is that *no constraint shares a key — and hence a scale or gradient clip — with a policy-gradient term.* The world-model losses (reconstruction, reward, continuation, dynamics/representation KL) are standard DreamerV3 and untouched. The hierarchy adds:

Goal autoencoder (key `goal_autoencoder`): reconstruction of the latent target, a KL to a uniform code prior, and the structural term of Eq. (4) — weighted by fixed w_s or by the adaptive multiplier λ_s of the pilot variant:

$$\mathcal{L}_{\mathrm{AE}} = \big\|\mathrm{dec}(\mathrm{enc}(s)) - s\big\|^2 + \beta\,\mathrm{KL}\big(\mathrm{enc}(s)\,\|\,\mathrm{uniform}\big) + \lambda_s\,\mathcal{L}_{\mathrm{struct}}, \qquad \beta = 0.25. \tag{5}$$

Manager actor (key `mgr_policy`): REINFORCE over decision steps on a normalized two-critic advantage, plus entropy *constraints* on both output heads — these are also adaptive multipliers, holding each head's normalized entropy near 0.5 rather than adding a fixed bonus:

$$\mathcal{L}_{\mathrm{mgr}} = -\,\mathbb{E}\Big[\log \pi^{\mathrm{mgr}}(z_t, m_t, d_t \mid \cdot)\,\hat{A}_t\Big] + \sum_{h\in\{\mathrm{skill},\,\mathrm{dur}\}} \lambda_h\big(\tau_H - \bar{H}_h\big), \qquad \hat{A}_t = \mathrm{norm}\big(A_t^{\mathrm{extr}} + 0.1\,A_t^{\mathrm{expl}}\big), \tag{6}$$

with separate critics (and replay-value twins) for the extrinsic and exploration returns. The sparsity and duration constraints sit in their own keys exactly as in Eq. (15): `mask_sparsity` carries $\lambda_m(\bar{p} - \tau_m) + \lambda_H(\tau_H - \bar{H}_{\mathrm{block}})$ and `goal_duration_prior` carries $\lambda_d(\mathbb{E}[d] - \tau_d)^2$.

Worker actor-critic (keys `wkr_policy`, `wkr_goal_value`, + replay twin), where the countdown enters. Let n_t be the number of steps left on the current goal *including* the current one ($n_t = d$ at the decision step, then decrementing), and

$$c_t \;=\; \frac{2\,\min(n_t,\,d_{\max})}{d_{\max}} - 1 \;\in\; [-1, 1], \qquad d_{\max} = 16\ (\text{var-}K)\ /\ K\ (\text{fixed}). \tag{7}$$

With `worker_timed_goals` the worker's policy *and* critic are conditioned on it: $\pi^{\mathrm{wkr}}(a \mid \mathrm{deter}, \mathrm{stoch}, g, c_t)$ and $V^{\mathrm{wkr}}(\mathrm{deter}, \mathrm{stoch}, g, c_t)$, identically at training and acting time (imagination rollout, imagination losses, replay value, and the real environment step all thread the same counter). The worker reward and return are

$$r_t^{\mathrm{wkr}} = \cos_{\max}\big(g_t,\,\mathrm{deter}_t\big), \qquad G_t = \lambda\text{-return}\big(r^{\mathrm{wkr}},\,V^{\mathrm{wkr}}\big)\ \text{reset at goal switches}, \tag{8}$$

and the losses are standard REINFORCE with an entropy bonus $(-\log\pi \cdot \mathrm{sg}(\hat{A}^{\mathrm{wkr}}) - \beta_H H)$ and the critic NLL against G_t. The reset in Eq. (8) makes each hold window its own value segment (Director's `split_traj`); the point of Eq. (7) is that this truncation is only well-posed if the worker can *see* where the

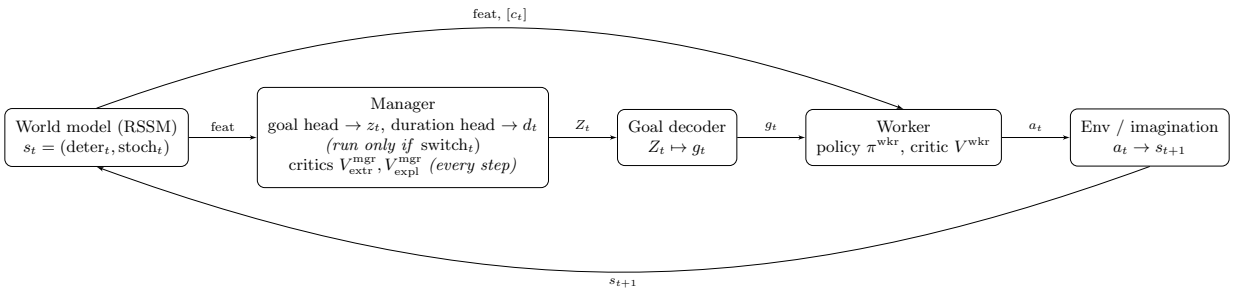

Figure 1: Data flow for one step under var-K. All boxes are learned networks; the countdown n_t that gates the manager (Eq. 9) is a register, not shown as a box. The manager's two heads run, and Z_t is overwritten, only on decision steps (switch$_t$=1); otherwise Z_t just holds its previous value (Eq. 10). The manager's two critics, like the goal decoder, worker, and world model, run every step. $[c_t]$ is the optional countdown input to the worker (timed subgoals, Sec. 3.3); without it the worker sees only (s_t, g_t).

segment ends. Without c_t, the same (state, g) pair carries return targets computed over horizons of 1–16 steps drawn from the manager's duration head — irreducible variance injected into the critic. With c_t, $V(\cdot, c_t)$ is a deterministic function of an observable horizon, and the policy can calibrate effort to the time budget (attempt near goals when c_t is low, commit to far goals when high). No term of any loss changes; only the two worker function approximators gain an input.

3.5 The combined recipe

"Combined recipe" below always means: Director base (Sec. 1) + masked edits (Eq. 1) + variable durations + the three controllers of Sec. 3 (edit-rate 0.3, mask entropy 0.5, duration τ_d with tolerance 0.1) + structural loss w_s=200. "Variable-K only" means variable durations with a weak fixed duration prior (weight 0.01) instead of the Lagrangian, no masking, w_s=0. "Mask only" means masked edits at fixed K=8, w_s=200.

4 Variable-duration option: mechanism reference

This section gives a self-contained account of how variable durations ("var-K") are implemented, independent of masking: every network's inputs, outputs, and loss, and the (non-network) counter that ties them together. Masking (Eq. 1) composes with this mechanism but is not required by it — "plain var-K" below means this section's machinery with the manager's goal head left as a full overwrite ($Z_t = z_t$ on a decision step), no mask head, w_s=0.

4.1 Components

Beyond the base agent (Sec. 1), var-K adds one network output (a duration head on the manager) and one piece of non-network state (a countdown register). Fig. 1 shows the resulting data flow for one environment/imagination step t.

Network	Runs	Input	Output	Trained by
Manager (goal head)	decision steps	feat(s_t) [+cond.]	$z_t \in \{1..C\}^L$	$\mathcal{L}_{\text{mgr}}$ (Eq. 6)
Manager (duration head)	decision steps	feat(s_t) [+cond.]	$d_t \in \{1..16\}$	$\mathcal{L}_{\text{mgr}}$ + `goal_duration_prior`
Manager critics (×2)	every step	feat(s_t)	$V^{\text{mgr}}_{\text{extr}}, V^{\text{mgr}}_{\text{expl}}$	NLL vs. λ-return
Goal decoder	every step	Z_t	g_t (goal in deter space)	$\mathcal{L}_{\text{AE}}$ (Eq. 5), not by $\mathcal{L}_{\text{mgr}}$
Worker actor	every step	feat(s_t), g_t, $[c_t]$	a_t	Eq. 8 REINFORCE
Worker critic	every step	feat(s_t), g_t, $[c_t]$	V^{wkr}	NLL vs. Eq. 8 G_t

Table 1: Every network touched by var-K: what it consumes, what it emits, and which loss trains it. "[+cond.]" = optional extra manager-conditioning inputs (`mgr_cond_*` flags), off in plain var-K. "$[c_t]$" = optional countdown input, off unless `worker_timed_goals`. Goal decoder gradient reaching the manager is a separate, orthogonal mechanism (`goal_reuse_*`, Sec. 3.4); the base goal decoder itself is trained only by the autoencoder loss, never by $\mathcal{L}_{\text{mgr}}$.

4.2 Decision timing

Let $n_t \in \{1, \ldots, d_{\max}\}$ be the steps remaining on the goal held at step t, including t itself (already used as the worker's countdown input, Eq. 7). A decision (manager forward pass) occurs at t iff the previous step was the last held step of its goal:

$$\text{switch}_t = \mathbb{1}[\, n_{t-1} = 1 \,], \qquad \text{switch}_1 := 1 \text{ (also forced at episode reset).} \tag{9}$$

On a decision step the manager samples z_t and d_t and the counter reloads; otherwise the running goal and counter both hold:

$$Z_t = \begin{cases} z_t & \text{switch}_t \\ Z_{t-1} & \text{else} \end{cases} \qquad n_t = \begin{cases} d_t & \text{switch}_t \\ n_{t-1} - 1 & \text{else.} \end{cases} \tag{10}$$

The duration head is a categorical over $n_{\text{dur}} = d_{\max} - d_{\min} + 1$ classes ($d_{\min}$=1, $d_{\max}$=16 by default); its own normalized entropy is held near a target (`manager_actent_duration_target`, same dual-ascent form as Eq. 3) so it cannot collapse onto a single duration class before the duration prior/Lagrangian has a chance to shape it. $g_t = \text{dec}(Z_t)$ is recomputed from Z_t every step (cheap decode), not cached, except that the *worker-visible rendering* of g_t (used for goal-conditioned image reconstructions/videos, not for any loss) is cached and refreshed only on $\text{switch}_t \vee \text{reset}_t$, matching Eq. 10.

4.3 Manager credit assignment: two resolutions

Because d_t varies, the manager's decision boundaries are not evenly spaced, unlike fixed-K Director. Two credit-assignment modes are implemented, selected by `variable_goal_block_rew`:

Full-resolution (default, used by every plain-var-K run in Sec. 6). The manager critics bootstrap on *every* imagined step, with continuation term $= 1 - \text{con}_t$ exactly like the flat/worker critic — $\bar{K}$ has no effect on how many value targets exist. Only the REINFORCE term of $\mathcal{L}_{\text{mgr}}$ (Eq. 6) is masked to decision steps, via switch_t: non-decision steps contribute a value-function target but no policy gradient.

Block-pooled (`variable_goal_block_rew=True`, Director-style option credit). Rewards are pooled over each *realized* hold segment and the whole manager problem is downsampled to one row per decision, mirroring fixed-K Director's `abstract_traj`:

$$\hat{r}_k^{\text{mgr}} = \text{agg}_{t \in \text{seg}(k)} \Big(r_t \textstyle\prod_{t' \le t,\, t' \in \text{seg}(k)} \text{con}_{t'} \Big), \qquad \text{agg} \in \{\text{mean}, \text{sum}\}, \tag{11}$$

where $\text{seg}(k)$ is the k-th realized hold window (length d_{t_k}, not a constant). `mean` makes the manager's per-decision return invariant to how finely time is partitioned (mirroring Director's own per-window mean); `sum` instead couples return to hold length, giving the manager a direct return-side incentive toward longer holds (`mgr_reward_agg`). Both the world-model features and the pre-edit conditioning code are downsampled to the same $\text{seg}(k)$ boundaries before the manager loss is recomputed on them (`downsample_at_switch_mask` in the code).

4.4 Duration prior / Lagrangian

The duration term of the manager loss (already given generically in Eq. 15) is, spelled out for var-K specifically,

$$\mathcal{L}_{\text{goal\_duration\_prior}} = \lambda_d \left(\mathbb{E}[d] - \tau_d \right)^2, \qquad \lambda_d \text{ stepped by Eq. (3) with } x = \mathbb{E}[d],\ \tau = \tau_d, \tag{12}$$

Here $\mathbb{E}[d]$ is the switch-weighted realized mean duration and the controller's tolerance is tol=0.1; this is the *Lagrangian* mode (`goal_duration_lagrange`). A simpler fixed-weight *regularizer* mode (`goal_duration_reg`, weight 0.01 in "plain var-K", Sec. 6) uses the same squared-deviation penalty at a constant weight instead of Eq. 3's adaptive λ_d. Both modes are independent of the mask-side controllers — turning masking off (as plain var-K does) removes `mask_sparsity` entirely and leaves `goal_duration_prior` as the only manager-side constraint term.

4.5 Status: not attributed to an implementation defect

Isolated post-fix re-tests of exactly this mechanism (`RECIPE=plain_vark`, Lagrangian mode, $\tau_d \in \{4, 8\}$, all four tasks, e296–e303) reproduce the same qualitative split reported for the pre-fix mechanism in Sec. 6 (finding 4) and for masking under the dual-head design (finding 4/F17): hopper stays at the dead reward-rate floor at both τ_d (e296/e297, indistinguishable from 20+ prior masked/var-K cells); acrobot

decays from an early peak toward the same floor without reaching it by ∼55% of budget (e298/e299); cartpole and cheetah are both clearly alive at both τ_d, with τ_d=8 consistently ahead of τ_d=4 (cartpole 748.7 vs. 601.0; cheetah 184.9 vs. 97.2, still rising) — alive but below the fixed-K=8 Director cheetah baseline (Table 3, ≈300–435), i.e. var-K alone is not catastrophic on cheetah but is not yet a free win either. Three points argue against an implementation bug specifically: (i) the failure is stable across two independent code revisions (the pre-2026-07-10 forward-fill/controller code, e166–e169/e178/e179, and the post-fix code reported here), with the same clean single-flag diff (var-K on vs. off, all else equal) reproducing return → 0 on hopper both times; (ii) the leading mechanistic hypothesis for *why* — the worker cannot observe its random per-goal deadline, injecting horizon variance into its critic (Sec. 3.3) — was tested directly by adding the countdown input: it rescues a collapsing *dense*-task var-K run 5.4× (cartpole, e187) but leaves hopper (e185) and acrobot (e192) exactly as dead as without it, so the failure is not simply an unpatched, known-fixable observability gap; (iii) the same sparse tasks are known to be solvable by an unrestricted manager (Director baseline, and the single-head design of finding 4/F17 at fixed K=8), so the failure is specific to *this* mechanism's interaction with sparse reward, not a general breakdown of the training pipeline. The single-head credit-assignment fix (F17) and var-K have not yet been tested *together* on hopper/acrobot — every single-head result to date (e214–e233) used fixed K=8 — so whether that fix also covers the switch-timing mechanism, or whether a second, duration-specific fix is needed, is an open, currently untested combination rather than a resolved negative result.

5 Experimental setup

Tasks are DeepMind Control from pixels (+proprioception): cartpole swingup and cheetah run (dense reward), hopper hop and acrobot swingup (sparse/impulsive reward). Two scales (Table 2); runs are 4M environment steps, single seed (replication is acknowledged, pending, and flagged wherever a claim rests on one run). Experiments carry global numbers (`e`n) recorded in the project log, where hypotheses, quantitative expectations, and outcome branches are *pre-registered before results*; scores are episode returns (max 1000), reported as last-10-episode means and best trailing-10 window ("peak").

Scale	Image	Model	Batch	Hardware
small	32^2	6M params	16×32	1×V100
BIG	64^2	Director-matched (deter 1024, 512-wide MLPs)	16×64	1×A100-80GB

Table 2: Run scales. BIG matches the published Director architecture; all cross-recipe comparisons are within-scale.

6 Results to date

Task	Recipe (BIG unless noted)	last10	peak	Run
cartpole	combined recipe (τ_d=4)	776	844	e171 (repro of e157: 833/8
cartpole	mask only, fixed K=8	360	419	e163
cartpole	variable-K only, τ_d=4, w_s=0	97	658	e167 (rose, then collapsed
cartpole	variable-K + struct, τ_d=8 (small)	725	758	e46
cheetah	combined, Lagrangian controllers	465	508	e175
cheetah	combined, fixed penalty weights	253	–	e142
hopper	mask only, fixed K=8 (no variable duration)	0	4.5	e164/e165
hopper	*any* masked / variable-K variant	0	∼6	20+ runs
hopper	Director baseline, fixed K=8	180	326	e124
acrobot	combined recipe	2	26	e176/e177
acrobot	Director baseline	learning (3.3M: 197, peak 295, still rising)		e180, in progress

Table 3: Key results. Single seeds; returns are episode scores (max 1000).

1. The components interact; neither suffices alone. On cartpole, masked edits at fixed K=8 and variable durations without masking/struct both fall far short of the combined recipe (Table 3, Fig. 2 left). This matches program history: masked fixed-K=8 variants collapsed or plateaued repeatedly, and plain

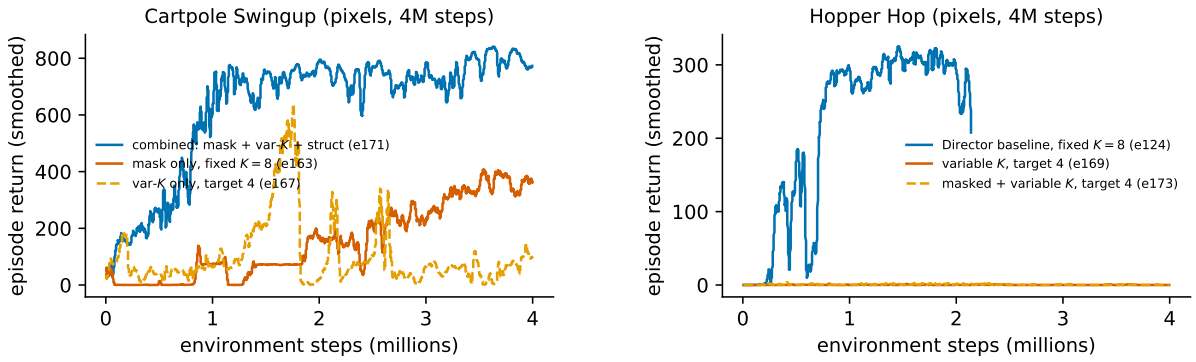

Figure 2: Left: on cartpole the combined recipe outperforms either component alone. Right: on hopper the Director baseline (fixed K=8, full goal replacement) learns, while the variable-duration variant with target 4 — differing *only* in the variable-duration package — never encounters reward; adding masking does not change this.

variable-K has only worked with both a duration prior and the structural loss (724, e46). The combined recipe achieves blk/step $\approx$ 0.6–1.1 at returns of 776–844 (Director: blk/step = 1).

2. Two-sided targets, then Lagrangian enforcement — the staged rescue of large-scale performance. Director-scale performance was recovered in two steps, each changing only how the manager's edit rate and duration are regularized. The three formulations, in the order they were tried:

Stage 0 — priced one-sided costs (underperforms). The first BIG runs added fixed costs to the manager objective and left duration free,

$$\mathcal{L}^{(0)}_{\text{mgr}} \;=\; \mathcal{L}_{\text{RL}} \;+\; c_{\text{sw}}\,\mathbf{1}[\text{switch}_t] \;+\; c_{\text{edit}}\,\|m_t\|_1. \tag{13}$$

Both penalty terms are *one-sided*: their gradient pushes editing and switching down with constant sign, and the only opposing force is the task advantage through a sampled, stop-gradiented mask — too weak to balance. The system therefore has no interior fixed point and runs to corners: the mask freezes ($\|m\|\to 0$, goal never updated, return $\to$ 0) or saturates, and the duration head drifts to near-maximal holds ($\bar{K} \approx 15$–21; the manager stops re-editing and goals go stale). Outcome: cartpole 528, cheetah 346, below the 435 baseline.

Stage 1 — fixed two-sided targets (recovers cartpole only). Reinstating explicit setpoints,

$$\mathcal{L}^{(1)}_{\text{mgr}} \;=\; \mathcal{L}_{\text{RL}} \;+\; w_d\left(\mathbb{E}[d]-\tau_d\right)^2 \;+\; \lambda_m\,(\bar{p}-\tau_m), \tag{14}$$

restores an interior equilibrium (the quadratic pulls from both sides). The two sides are structurally *asymmetric* here: the mask term $\lambda_m(\bar{p}-\tau_m)$ was already dual-ascent *and* already lived in its own independently-scaled loss key, whereas the duration quadratic is folded directly into the manager's policy loss, sharing its scale and gradient clip. That asymmetry is where both remaining problems live. First, w_d is scale-fragile: early in training $(\mathbb{E}[d]-\tau_d)^2$ can be $\sim$64, so w_d=0.1 contributes an $O(1+)$ loss that dominates the percentile-normalized REINFORCE term it shares a scale with — the manager optimizes the prior instead of the task ("magnitude domination"); only $w_d \le 0.03$ coexists. Second, the workable w_d is task-specific and does not transfer. The mask side, already separated, exhibits neither problem. Outcome: cartpole 709, cheetah 253.

Stage 2 — same targets, symmetric dual-ascent enforcement (recovers both). The fix makes the duration side structurally identical to the mask side: the fixed weight becomes a multiplier updated by Eq. (3), and the duration term moves *out* of the policy loss into its own independently-scaled loss key — mirroring where the mask term already was — so no constraint shares the policy gradient's scale or clip. A per-block entropy floor is added on the mask:

$$\mathcal{L}^{(2)}_{\text{mgr}} \;=\; \mathcal{L}_{\text{RL}} \;+\; \underbrace{\lambda_d\left(\mathbb{E}[d]-\tau_d\right)^2}_{\text{key: goal\_duration\_prior}} \;+\; \underbrace{\lambda_m\,(\bar{p}-\tau_m) \;+\; \lambda_H\left(\tau_H-\bar{H}_{\text{block}}\right)}_{\text{key: mask\_sparsity}}, \tag{15}$$

where λ_d is stepped on the switch-weighted deviation $|\bar{\mathbb{E}}[d]-\tau_d|$ against a tolerance of 0.1 (symmetric, so it relaxes from either side) while the penalty it scales is the per-decision squared deviation; $\bar{p}$ is the mean edit probability; and $\bar{H}_{\text{block}}$ the per-block mask entropy whose floor τ_H prevents collapse onto

always/never-edited blocks. Each λ grows only while its constraint is violated and decays otherwise, so enforcement strength is learned per task while the *setpoints stay fixed*. Mechanism metrics confirm the design: duration std $\sim$20$\times$ tighter than Stage 1 (0.0012 vs 0.024), mask pinned at 0.31–0.35, and at convergence $\lambda_d \to 0$ — the constraint holds with no active pressure. Outcome: cartpole 833, cheetah 465–508.

The converse design — fully adaptive targets with no fixed setpoints — kept statistics interior but learned nothing off cartpole. The transferable recipe is therefore a *fixed target with an adaptive multiplier*: the target supplies the missing gradient direction (Stage 0's failure), the multiplier supplies the task-appropriate magnitude (Stage 1's failure).

3. Sparse-reward tasks fail under every restricted variant. On hopper, every masked and/or variable-duration configuration tested (16+ runs across recipes, scales, controllers) shows manager extrinsic reward $\approx 10^{-4}$ throughout: reward is never encountered, so nothing downstream can learn. The Director baseline finds reward within 0.2–0.4M steps. The same baseline is now visibly learning acrobot while the combined recipe stays at zero there — the failures are caused by our restrictions, not by task difficulty for the base hierarchy.

4. Two independently sufficient restrictions kill hopper — variable duration and masking alone, separately. Resolved-configuration diffs show the variable-K-only hopper run and the Director baseline differ *only* in the variable-duration package (duration head, weak prior toward τ_d=4) — and that difference separates 180 (peak 326) from exactly 0 (Fig. 2 right). Raising the target to 8 alone lifts the extrinsic reward rate from $\sim 10^{-4}$ to a noisy 10^{-3}–10^{-2} band that never sustains above the alive threshold through 3.2M steps — real but weak, and not rising further; duration-return coupling via sum aggregation did not help either (dead at 0.6M, cancelled). Separately, mask only at a *fixed* K=8 — the same schedule as the alive Director baseline, with masking as the only change — is also dead (0/4.5, Table 3). Since neither restriction needs the other to break hopper, hold length is not the single root cause we initially framed it as: masking and variable duration are each, independently, sufficient to remove sparse-task reward, and the surviving common factor is the unrestricted Director hierarchy itself. This reframes the search: rather than attributing the failure to one axis, the open question is what *any* deviation from whole-code, fixed-schedule goal replacement removes that sparse exploration depends on.

5. The collapse anatomy points at code drift and worker unreliability. In the variable-K-only cartpole run the manager's advantage grew to 4$\times$ its healthy level, then crashed to ≈ 0 permanently — at the same moment the worker's goal-reaching reward dropped to its run-low (0.14–0.21 vs. 0.32–0.46 in stable runs) and the goal autoencoder's reconstruction error kept climbing with *no structural anchor* (w_s=0). Policy entropies stayed pinned at their targets throughout — the collapse is a manager–worker–autoencoder coordination failure, not an entropy effect. Two repairs are under test: the structural loss as a code stabilizer (Sec. 3.2), and making the worker's horizon observable (Sec. 3.3).

6. A single-head masked manager rescues hopper, refining Finding 4 (2026-07-15, preliminary). Finding 4 framed the open question as "what does *any* deviation from whole-code, fixed-schedule replacement remove" — implying every restriction breaks sparse exploration. This no longer holds unconditionally. All prior masked variants used two independent heads per block (which content to write, and whether to write it), trained by separate REINFORCE estimators; whenever a block is not edited, the content head's gradient for that block is discarded, an asymmetry we call the *discarded-content confound*. Replacing the two heads with one (C+1)-way categorical per block (class 0 = abstain, keep $Z_{t-1}[i]$; classes 1..C = overwrite, Sec. 2) removes this confound by construction. On the fixed-K=8 Director base with this single-head mask left fully free (no sparsity target imposed), BIG-scale hopper reaches extrinsic reward rate 0.245 (versus the 10^{-4}–10^{-2} band of every Finding-3/4 cell) and a peak 280–290 last-15 plateau around 1.6–2.1M steps, ahead of the Director baseline's own 196 (peak 326) at a comparable point (Fig. 3, bottom row). It reaches this *without* editing sparsely: the realized edit fraction stays at 0.94–0.97 throughout training, so the free manager simply never learns to abstain much. The rescue therefore isolates to the credit-assignment mechanism, not to any change in how much of the goal gets rewritten — a distinction Finding 4 could not draw, since every prior masked cell coupled the confounded credit assignment to whatever edit rate its penalty enforced. Imposing sparsity still costs performance under the new design: pairing the single head with the structural loss roughly halves the peak score (165 vs. 302) though, unlike the old design, does not kill it outright; forcing the edit fraction down via the mask-sparsity-target ratchet (Sec. 7.2) keeps the run flat near zero for its full 4M-step budget, and combining both constraints reproduces

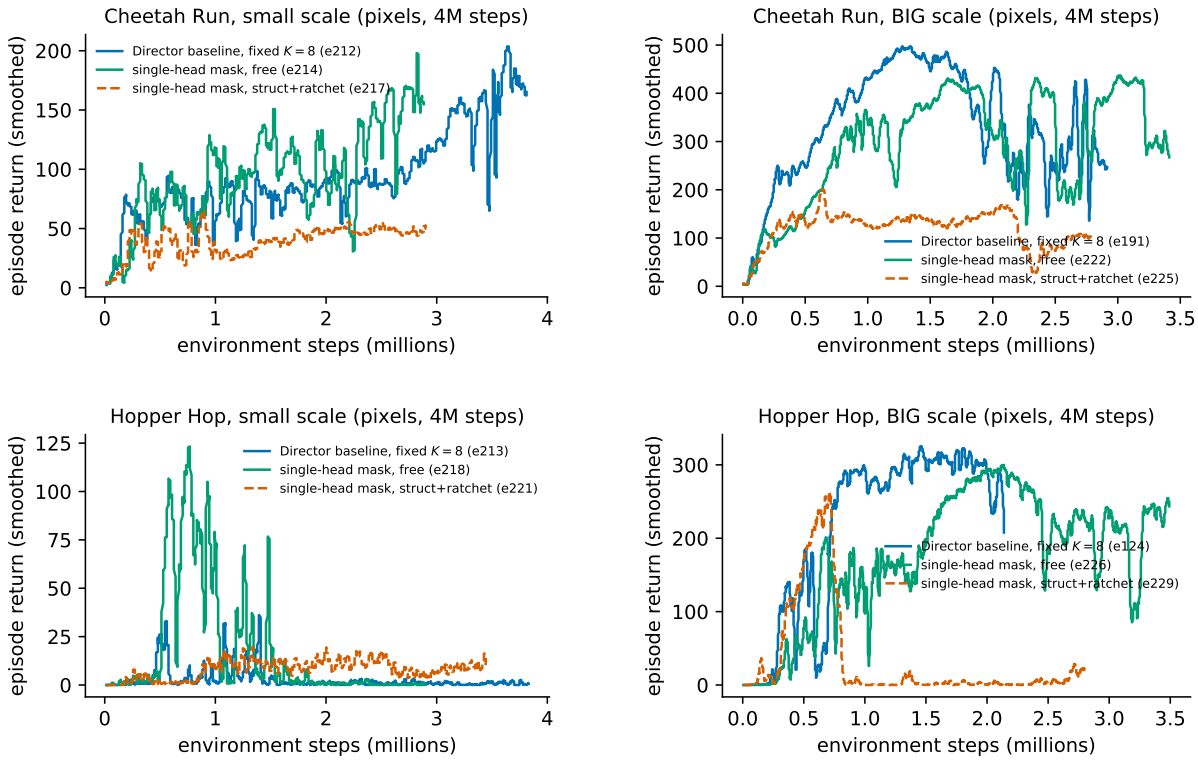

Figure 3: Director baseline vs. two single-head masked-manager cells, one panel per env × scale: free (S0R0, no struct, no ratchet) and struct+ratchet combined (S1R1, struct-adapt target 0.006, ratchet target 0.3 — the original single-head campaign's setpoints, since revised and retested in Sec. 7). Top: cheetah (dense-hard) at small (e212 vs. e214 vs. e217) and BIG (e191 vs. e222 vs. e225) scale — the free cell matches or exceeds Director at both scales with no late decline, while S1R1 plateaus 2–3× lower. Bottom: hopper (sparse-hard) at small (e213 vs. e218 vs. e221, all near the dead floor) and BIG (e124 vs. e226 vs. e229) scale — the free cell opens a substantial lead over Director mid-training before declining in its final third (see the correction above Finding 6), while S1R1 climbs alongside the free cell early (to ∼208 by 0.5M) then collapses outright once the ratchet's target fully takes hold. S1R1's BIG-scale curves (e225, e229) are complete/archived; all others are single seeds, smoothed over a trailing 20-episode window, plotted to their latest logged episode as of 2026-07-15 (still training).

Finding 5's collapse signature (a promising early climb to 208 by 0.5M, then a permanent crash). The rescue is BIG-scale only so far: every small-scale single-head hopper cell rises transiently and then decays, matching the small Director control's own dead floor — consistent with hopper needing Director-matched scale independent of the manager design.

Correction (2026-07-15, same day as the reading above): the BIG-scale hopper rescue is not yet stable. Continued training past the point the "alive" reading above was based on shows both the free cell and its struct-paired sibling *decline* substantially in large-window average after their peaks — free: peak ∼290 around 1.9M steps down to ∼75 over the last ∼270-episode window at 3.2M; struct-paired: peak ∼150 down to ∼24 — visible directly in Fig. 3's bottom-right panel. This is not Finding 5's collapse signature: the manager's extrinsic advantage stays small throughout (no advantage spike) and the worker's goal-reaching reward is still *rising*, not dropping, over the same window, so the usual coordination-failure diagnosis does not apply. The cheetah cells (Fig. 3, top row) show no equivalent decline over the same training window. Read Finding 6 as "the single-head design rescues hopper for roughly the first two-thirds of training, then something currently unexplained erodes the gain" rather than as a settled result; root-causing the late decline is the immediate next step before any claim here is finalized. Single seed throughout; details in Sec. 7.

7 Experiments in progress

7.1 Five mechanisms for goal-code sparsity, at a glance

Sections 7.2–7.5 introduce five distinct ways to make the manager reuse more of the previous goal per decision. They differ in exactly one thing: *where the sparsity signal gets a gradient into the network*. Table 4 summarizes all five before the detailed treatment; the short version is that three are ordinary differentiable losses, one is a REINFORCE reward penalty (and the weakest of the five, F18), and one is not a loss at all but a change to how the goal code is sampled. All three differentiable losses (mechanisms 1, 4, 5) turn out to share mechanism 2's failure mode once their target is pushed high enough (F19, §7.6) — differentiability alone does not avoid it.

Mechanism	What it controls	How pressure is applied	Gradient path
1. Abstain (§2, `mask_sparsity`)	mean non-abstain probability $\bar{p}$	loss $\lambda_m(\bar{p} - \tau_m)$, Eq. (15)	direct: backprop through a softmax probability, Eq. (19)
2. Implicit (§7.2, `impl_sparsity_mode`)	s_t, measured post hoc: did the code repeat?	reward penalty $\lambda(1 - s_t)$ on r_t^{extr}, Eq. (17)	REINFORCE only: $\log \pi^{\text{mgr}} \cdot \hat{A}$ (Eq. 6); no direct path, s_t is stop-gradiented (Eq. 16)
3. Delta (§7.3, `goal_delta_mode`)	nothing on its own — no loss key	resamples from p_t = normalize(p_{t-1} + votes), Eq. (20)	none of its own; gives mechanisms 2, 4 or 5 an explicit "keep" action to attach a gradient to
4. Goal-space reuse (§7.4, `goal_reuse`)	decoded-goal similarity u_t	loss $\lambda_r(\tau_r - u_t)$, dual-ascent as Eq. (3), on Eq. (21)	direct: straight-through gradient through the sampled code, decoder's own weights frozen
5. Soft block overlap (§7.5, `goal_soft_reuse_adapt`)	block-overlap o_t between successive plain softmaxes	loss $\lambda_o(\tau_o - o_t)$, dual-ascent as Eq. (3), on Eq. (22)	direct: backprop through both steps' own softmax, no sampling and no decoder pass involved

Table 4: Five mechanisms tried for promoting goal-code sparsity, in the order they were introduced. Mechanisms 1, 4 and 5 are ordinary losses added to $\mathcal{L} = \sum_k s_k \mathcal{L}_k$; mechanism 2 instead reshapes the manager's *reward* and relies on the policy-gradient estimator to carry the signal — the same channel task return uses, and the one Finding 1/F18 show rails and kills learning under pressure. Mechanism 3 is plumbing, not a pressure source: it composes with 2 (e250–e257, testing whether a real "keep" action rescues the REINFORCE penalty — cancelled without a readout, superseded by mechanism 4) and with 4 (e258/e260/e262/e264, tested at scale, §7.4; the `goal_delta_mode` counterparts e259/e261/e263/e265 were cancelled and replaced by e274–e277, a manager-input ablation removing the raw code channel). Mechanisms 4 and 5 both reach the magnitude-domination collapse of Finding 1/F18 once their target fully engages (F19, §7.6), despite a fully differentiable gradient path in both cases.

7.2 From explicit masking to *implicit* sparsity

The restrictions of §2 force sparsity by *construction* (a per-block edit/abstain mask). This proved to be only a proxy for the real quantity of interest—how much of the goal the manager *chooses* to keep unchanged—and the discarded-content bookkeeping of a hard mask complicates credit assignment. We drop the mask entirely: the manager regenerates the whole goal code Z_t each decision (plain Director), conditioned on the previous code Z_{t-1} so it can *deliberately* re-emit blocks, and we simply *measure* the realized reuse.

Implicit sparsity. At each manager decision the fraction of goal-code blocks whose class is unchanged from the previous goal is

$$s_t \;=\; \frac{1}{L}\sum_{i=1}^{L} \mathbf{1}\big[\arg\max Z_t^{(i)} = \arg\max Z_{t-1}^{(i)}\big] \;=\; \mathrm{sg}(s_t) \quad \text{(no gradient, by construction)}, \tag{16}$$

(1 = regenerated identically, 0 = every block changed); the $\mathrm{sg}(\cdot)$ is not decorative — $\arg\max$ and $\mathbf{1}[\cdot]$ have zero gradient almost everywhere, so s_t is a pure measurement with no path back into the network that produced Z_t. A continuous analogue, $s_t^{\mathrm{cont}} = \frac{1}{2}(1+\cos_{\max}(g(Z_t), g(Z_{t-1}))) \in [0,1]$ (decoded-goal similarity), is logged alongside it. Both are logged for every run (`goal/implicit_sparsity_block`, `..._cont`), averaged over manager-decision steps (the seed step is excluded).

An implicit-sparsity controller. With no gradient available through s_t itself, the only remaining lever is to shape the manager's *reward* and let REINFORCE carry the signal:

$$r_t^{\mathrm{shaped}} \;=\; r_t^{\mathrm{extr}} \;-\; \lambda\,(1 - s_t), \qquad \lambda \leftarrow \mathrm{clip}\big(\lambda \cdot \exp(\eta\,(\bar{s} - s_t)),\, \lambda_{\min},\, \lambda_{\max}\big), \tag{17}$$

the same dual-ascent rule as Eq. (3), driving the measured kept-fraction toward target $\bar{s}$. *Direct* fixes $\bar{s}$ from step 0; *ratchet* anneals $\bar{s} : 0 \to \bar{s}$ over training with one-sided λ (pressure never relaxes). Because s_t is stop-gradiented (Eq. (16)), every gradient this cost contributes to the manager reaches it only through the policy-gradient term $\log \pi^{\mathrm{mgr}} \cdot \hat{A}$ of Eq. (6), with $\hat{A}$ now built from r_t^{shaped} — sparsity pressure enters through the *same channel* as the task-return signal, rather than as its own direct gradient. This composes with the structural term (§3.2), which raises reuse *indirectly* via temporal code correlation and needs no reward-shaping at all.

Baselines. Measured at each trained Director's final checkpoint (Table 5), a plain Director *already* keeps 0.32–0.45 of blocks per decision unprompted (vs. $\approx 1/C$ early), with small models keeping more than BIG. The target $\bar{s} = 0.7$ (equivalently a 0.3 change fraction) is set above this band so the controller must actively increase sparsity.

Director baseline	scale	kept s	s^{cont}
e212 cheetah	small	0.445	0.971
e213 hopper	small	0.414	0.887
e123 cheetah	BIG	0.316	0.980
e124 hopper	BIG	0.367	0.849
e125 acrobot	BIG	0.395	0.770

Table 5: Implicit sparsity of trained Director baselines (final checkpoint). A BIG cartpole reference (e115) could not be measured—its pre-redesign masked checkpoint no longer shape-matches current code.

The e234–e249 matrix. Pure Director copy (fixed K=8, $\bar{s}$-conditioning on) at 1M steps, over {hopper, cheetah} × {small, BIG} × {struct-adapt only; ratchet→0.7; struct+ratchet; direct 0.7}. Question: which lever reaches $s \approx 0.7$ at least task-return cost—indirect (struct/code correlation) or the direct REINFORCE controller—and does raising sparsity above the Director's natural band help or cost return.

All 4M steps; "alive" on sparse tasks = extrinsic reward rate $> 10^{-2}$ by 1M steps (baseline reference). Cells cancelled at 0.6M for being flat while their comparators learned: sum aggregation (hopper), the combined recipe with τ_d=8 (hopper and acrobot), and 10× manager exploration (hopper).

Readout logic (pre-registered): if the countdown cells (e185, e187) lift their counterparts, the worker's unobservable horizon is a confirmed bottleneck and timed subgoals become part of the recipe. If struct-on-hopper (e189) kills the target-8 pulse, struct's locality face dominates on sparse tasks and it should be scheduled or gated, not constant; if it strengthens the pulse, struct is primarily a code stabilizer and belongs in every variable-K run, with the adaptive version (e195) replacing the hand-tuned weight.

Preliminary readout (2026-07-14, single seed, 50–83% of a 4M-step budget — all cells still training). Countdown's effect is split cleanly by task type, not uniform: on cartpole, e187 (plain var-K τ_d=4 + countdown) reaches 750/757 against e166's 130/192 on the byte-identical config without it — the "lift" branch firing decisively, confirming worker confusion about *when* the goal changes is a real, primary

Run	Configuration
e178	var-K τ_d=8, w_s=0
e179	var-K τ_d=8, w_s=0
e180	Director baseline
e185	e178 + countdown
e186	combined + countdown
e187	var-K τ_d=4 + countdown
e188	mask fixed-K=8 + countdown
e189	var-K τ_d=8 + struct 200
e190	Director baseline
e191	Director baseline
e192	var-K τ_d=8 + countdown
e193	var-K τ_d=4 + struct 200
e194	var-K τ_d=8, w_s=0
e195	var-K τ_d=4 + adaptive struct
e196–e199	full stack: combined + countdown + adaptive struct
e200–e203	full stack, τ_d=8, struct-adapt target 0.006
e204–e211	e200–e203 recipe + mask-sparsity-target ratchet (§7.2)
Single-head mask campaign (e212–e229; e194–e211 cancelled+archived 07-14 to free GPUs).	
e212–e213	pure Director baseline, fixed-K=8
e214–e229	**single-head (joint) mask** on the fixed-K=8 Director base, 2×2: struct{off, adaptive 0.006} × ratchet{off, on}
Looser-setpoint retest (e230–e233; e224/e225/e228/e229 cancelled+archived 07-15).	
e230–e233	single-head S1R1 (struct + ratchet together), ratchet target loosened 0.3→0.5 (same ∼1M-step anneal, recalibrated

Table 6: Hypothesis matrix. The small-cartpole 2×2 (e46/e166/e193/e194) disentangles duration target from struct; e178/185/189 plus a later struct×countdown cell factorize the hopper rescue; e190/e191 provide the Director-matched dense-task controls. A cartpole run of the combined recipe with w_s=0 (short-queue) probes struct within the full recipe.

cost of variable duration on dense tasks. On hopper, e185 stays statistically level with its non-countdown counterpart e178 (both in the collapse-zone on worker reliability, both at the reward-rate floor) — horizon observability is not hopper's binding constraint, so countdown is not a universal sparse-task fix. Struct shows both of its predicted faces at once, on different cells: e189 (struct 200 added to the τ_d=8 hopper pulse) makes the already-weak pulse *weaker*, firing the "locality face dominates, gate it on sparse tasks" branch; meanwhile e160 (struct removed from the full combined recipe on dense cartpole) collapses from a working peak of 138 down to 22 with manager advantage flatlining at zero, extending struct's stabilizer role from plain variable-K (already established) to the masked combined recipe as well. At τ_d=8 specifically, struct turns out to be dispensable on dense tasks, but only at the larger scale: e179 (BIG, no struct) is stable at 740/755, while the identical recipe at small scale (e194) stays stuck near 150 — struct's dispensability at longer holds looks scale-dependent, not general. Finally, acrobot's best-guess sparse recipe (e192) stays at the dead floor while its own Director-baseline control (e180) climbs steadily over the same window, splitting acrobot from hopper as a separate failure mode rather than a shared one.

Mask-sparsity-target ratchet (2026-07-14). Under the mask, the manager's exploration-reward advantage (a dense goal-deter reconstruction error, block-pooled the same way as the extrinsic reward) is a single scalar over the *entire* resulting goal, applied uniformly to every manager head — editing 1 of 8 blocks earns the same-magnitude exploration credit as editing all 8, with no per-block attribution of which edit produced the novelty. Since the sparsity Lagrangian (§7) already drives the edit fraction toward its target from the first training step, we hypothesized this compounds with F12 (masking alone already kills hopper/acrobot at fixed K=8) by starving goal-space exploration before the manager has any signal about which blocks are worth editing. We added an open-loop, velocity-capped ratchet on the sparsity Lagrangian's *target* itself (distinct from the already-adaptive Lagrange weight): it ramps from editing the whole goal (target 1.0) down to the sparse target (0.3) over approximately the first 1M environment steps, then holds. The mechanism is opt-in and a no-op unless explicitly enabled. e204–e211 were cancelled before producing results (hardware reallocated to the single-head campaign, Table 6); the ratchet was instead tested there as the "R1" cells of the single-head 2×2.

Ratchet readout (2026-07-15): the exploration-starvation hypothesis is refuted. The ratchet anneals exactly as designed — realized edit fraction tracks target down to 0.30 by ∼1.1M steps on both

cheetah and hopper — but every ratchet-on cell underperforms its free counterpart: on hopper, 22 (peak 52) versus the free cell's 214 (peak 302); on cheetah, 134 (peak 143) versus 181–205 (peak 434–455) for the free and struct-on cells. Left free, the manager does not choose to abstain (edit fraction stays at 0.94–0.97 the whole run, Finding 6), so there was no starved exploration signal to rescue in the first place; forcing the edit fraction down early removes the behavior that was working rather than adding the credit signal we hypothesized was missing. One confound this readout cannot rule out on its own: targets 0.3 (mask, R1) and 0.006 (struct-adapt) were both carried over unchanged from the old dual-head design, so the underperformance could be a too-tight-setpoint artifact rather than a fact about forced sparsity in general. e230–e233 (Table 6) retest the S1R1 cell at looser targets (mask 0.5, struct-adapt 0.01) on all four tasks to separate the two explanations.

Looser-setpoint retest (e230–e233): partial, but points toward a setpoint artifact. All four runs were cancelled 2026-07-15 at only 15–17% of their 4M-step budget to free hardware for the e234–e249 launch below, so none reached a final verdict. At cancellation: cartpole (e230) 435/479 and still climbing; acrobot (e231) 54/84, weak but the first non-negligible single-head acrobot result at any setting (every prior dual-head acrobot cell, e176/e177, was ≤ 2.5); cheetah (e232) 173/178, already above e224/e225's tight-setpoint plateau (∼110–140); hopper (e233) **215/216**, far above e228's final 22/52 and inside e226's alive-plateau range (280–286) at under a fifth of the training budget. The key cell (hopper) leans toward refuting "any imposed sparsity target collapses the task" at the looser setpoint, but e226/e227 themselves looked alive at a comparable fraction of budget before declining substantially by 3M+ steps (§7.2 correction below), so this is a lead, not a settled result.

The e234–e249 matrix: results. All 16 runs completed the full 1M-step budget (Table 7). The pre-registered question — which lever reaches high implicit sparsity at least task-return cost, struct or the direct REINFORCE controller — turned out not to be the operative one: **struct-only is safe in all 4 cells tried; every one of the other 12 cells, which add the REINFORCE controller in ratchet, direct, or struct+ratchet form, collapses to near-zero score regardless of task or scale.** Cheetah at BIG scale under struct-only (330/335) lands within noise of contemporaneous plain-Director controls at the same step count (≈250–305); hopper BIG (115/116) and cheetah small (118/159) are clearly alive under struct-only but well below their Director references, a real but partial cost; hopper small stays at the established small-hopper noise floor (Finding 15) regardless of condition. Every controller-bearing cell, by contrast, scores 0–8.6 and fails to even reach its own target: the dual multiplier λ rails to its configured ceiling (5.0) in all 12 cells, typically within the first third of training, while the realized change fraction plateaus at 0.44–0.52 — short of the 0.3 implied by the 0.7 kept-target. This is the same magnitude-domination failure mode as the duration-reg cliff (§3) and the one-sided edit/switch costs of Finding 2, and it reproduces the single-head mask ratchet's failure under the old design (e228/e229 above) under a third, independent mechanism: forcing a measured sparsity quantity via an aggressive per-step REINFORCE penalty destroys learning, while a softer or indirect lever does not. One qualification: struct's own kept-fraction (0.20–0.38) is not clearly *higher* than the unprompted Director baseline (0.32–0.45, Table 5) — on 3 of 4 cells it is lower, though that baseline was measured on checkpoints trained without `mgr_cond_goalcode`, so the comparison is suggestive rather than a controlled ablation. struct is established here as *safe*, not as a demonstrated sparsity lever; `agent.impl_sparsity_mode` stays off by default pending either a much weaker controller weight or a non-REINFORCE mechanism (e.g. a KL-to-prior penalty).

Cell	Task	Scale	score	kept s (final)	λ (final)
struct	hopper	small	1.9 (46.5)	0.36	—
struct	cheetah	small	117.8 (159.0)	0.26	—
struct	hopper	BIG	115.1 (115.7)	0.38	—
struct	cheetah	BIG	**329.8 (335.4)**	0.20	—
ratchet / direct / struct+ratchet	hopper	small	0.0 (0.5–0.7)	0.56	5.00 (railed)
ratchet / direct / struct+ratchet	cheetah	small	1.4–3.0 (4.2–5.0)	0.51–0.56	5.00 (railed)
ratchet / direct / struct+ratchet	hopper	BIG	0.0 (0.2–1.0)	0.49–0.53	5.00 (railed)
ratchet / direct / struct+ratchet	cheetah	BIG	3.4–4.1 (6.1–8.6)	0.52–0.55	5.00 (railed)

Table 7: e234–e249, full 1M-step results. Each ratchet/direct/struct+ratchet row summarizes 3 cells (individual condition scores differ by <5 points; see `EXPERIMENTS.md` for per-cell numbers). λ = `impl_sparsity_scale_mean`, the dual-ascent multiplier on the REINFORCE cost; its configured ceiling is 5.0.

Root cause: the implicit controller is architecturally forced into the one mechanism already known to fail. The two designs differ in exactly one place — where the sparsity gradient comes from — and that difference is visible directly in the two loss forms. Under the explicit single-head mask (`mode='prob'/'prob_entropy'`, used by e224–e229), abstain is class 0 of each block's own $(C{+}1)$-way categorical (§2), so the mean edit probability $\bar{p}$ of Eq. (15) is itself a smooth function of the network's logits, $p_i = 1 - \pi_i(\text{abstain})$, and the sparsity term

$$\mathcal{L}_{\text{sparsity}}^{\text{abstain}} \;=\; \lambda_m\,(\bar{p} - \tau_m) \;+\; \lambda_H\,(\tau_H - \bar{H}_{\text{block}}) \qquad \text{(Eq. 15)} \tag{18}$$

backpropagates directly into each block's own logit. Because p_i is a deterministic softmax output, not a sampled outcome, the chain rule gives an ordinary closed-form derivative,

$$\frac{\partial \mathcal{L}_{\text{sparsity}}^{\text{abstain}}}{\partial \,\text{logit}_{i,0}} \;=\; \frac{\lambda_m}{L} \cdot \underbrace{\big(-\pi_i(\text{abstain})(1-\pi_i(\text{abstain}))\big)}_{\partial p_i / \partial\,\text{logit}_{i,0}} + \lambda_H \cdot \frac{\partial \bar{H}_{\text{block}}}{\partial\,\text{logit}_{i,0}}, \tag{19}$$

available every step regardless of which class the block actually sampled, with zero sampling variance — exact, per-block credit, entirely independent of the REINFORCE/task-return channel, with a dedicated "keep" outcome one bit away. Eq. (17), by contrast, needs a full rollout sampled, an advantage estimated from it, and that estimator's variance absorbed before any sparsity signal reaches the network at all. The implicit controller has no such action: every block always redraws a real class, so "kept" is only ever the post-hoc, stop-gradiented measurement s_t of Eq. (16), and the sparsity cost can only reach the network via reward-shaped REINFORCE, Eq. (17). Contrasting Eq. (18) (ordinary supervised-style gradient, its own loss key, Eq. 15) against Eq. (17) (folded into the *reward*, sharing the manager's REINFORCE estimator with task return) makes the failure mode legible: this is structurally identical to the explicit-mask framework's own REINFORCE-reward-shaped mode, which Finding 1 already showed rails and kills reward in the earliest bring-up runs (e10/e11). Dropping the mask removed not only the discarded-content confound (the intended win of the pivot, §7.2) but also access to a differentiable sparsity proxy, forcing the new controller into the one mechanism already known to be broken. The raw trajectories are consistent with this account: implicit-controller cells are dead from step 0 (e.g. hopper BIG ratchet: score ≈ 0 throughout, no initial rise), whereas the explicit-mask ratchet cells at least achieved partial or transient learning before degrading (e228: 22/52; e229: rose to 208 before collapsing at 500k) — the differentiable per-block loss shaped behavior carefully from the start; the reward-mediated scalar penalty never had the chance to.

7.3 Delta goal generation: a differentiable reuse action

The root-cause analysis above implies a fix: give plain-Director goal generation a differentiable way to express "keep this block", rather than only being able to measure reuse after the fact. We introduce `goal_delta_mode`: the manager's per-class logits pass through an *unnormalized, independent* sigmoid ("votes" — deliberately not a softmax, so every class can be driven toward 0 *simultaneously*, which a normalized distribution can never do) that is *added* to the previous goal code's own pre-sample distribution before the categorical sample:

$$\text{votes} = \text{clip}(\sigma(\text{logits}),\, 0,\, c), \qquad p_t = \text{normalize}\big(p_{t-1} + \text{votes}\big), \tag{20}$$

applied independently per block ($L{=}8$ blocks of $C{=}8$ classes by default; no cross-block coupling in the combination itself, only whatever correlation the shared network trunk that produces the logits learns to encode). c (`goal_delta_clip`, default 1, a no-op at sigmoid's own ceiling) caps a single vote. p_{t-1} is the PREVIOUS decision's own pre-sample distribution, not its collapsed sample: a one-hot is a lossy view of that decision (a 55%-confident pick and a 99%-confident pick both collapse to an identical one-hot once sampled), so basing next-step stickiness on the one-hot would silently max out trust at 1 regardless of how contested the choice actually was, capping every single-vote switch at a coin-flip tie no matter what. Using the real p_{t-1} instead makes stickiness scale with actual prior confidence: a decisively-won class ($p_{t-1} \approx 1$) still caps a fresh vote at a tie (votes are bounded in $[0, 1]$, same as the standing class's residual weight), but a contested class (e.g. 0.55/0.45) can be outright overtaken by one strong vote on the alternative ($0.45 + \sim 1 \gg 0.55 + \sim 0$) — a genuinely decisive switch in one decision, not just a coin flip. (The fallback for p_{t-1} when no real previous decision exists yet — episode-start seeding, report-time proposals — is the state encoding's one-hot, which is not a manager decision at all, so treating it as a firm commitment is the right default there.) All-≈ 0 votes for a block leave p_t numerically unchanged from p_{t-1}, so the same class stays most likely: an explicit, gradient-carrying reuse default, in contrast to arg max-based reuse measurement,

which carries no gradient by construction. Critically, actively re-voting the same class is still distinguishable from silent reuse: it shifts that class's margin over the alternatives, so any downstream loss reading p_t (rather than only the hard sample) can tell the two apart. `goal_delta_mode` implies `mgr_cond_goalcode` and is mutually exclusive with `use_masked_goals` (it is itself a reuse mechanism, replacing rather than composing with the explicit block mask). The same p_t is also fed back as the manager's own next-step conditioning input in place of the collapsed one-hot – a strictly richer signal (the one-hot is a lossy arg max of it) that additionally reports how *contested* the previous decision was.

Implementation-wise the change touches every site that samples or re-derives the manager's skill distribution: the online policy step, both imagination-rollout paths, the replay re-derivation used for the worker target, and — easy to miss, since it is a separate reconstruction of the same distribution purely for the loss — the actor-loss site that recomputes $\log \pi(\text{action})$ for the manager's REINFORCE term. Unless that site applies the identical delta combination, the policy gradient is taken under the wrong distribution. `goal_delta_mode` is currently smoke-tested only, on cartpole (three legs: delta off, delta on at clip 1.0, delta on at clip 0.5), all passing crash-free through the online policy step, both imagination-rollout paths, and the actor-loss re-derivation site, with no NaN/Inf in any logged scalar and a stable implicit-sparsity signal close to the delta-off baseline at random initialization (expected: p_{t-1} starts near-uniform, so early behavior resembles a fresh decision rather than an artificially sticky one); an e-numbered A/B against the existing `impl_sparsity_mode` cells (Table 7) is the natural next experiment.

7.4 Targeting the decoded goal directly, not code identity

Both the original `impl_sparsity_mode` controller and §7.3's delta mechanism operate at the goal-*code* level (block class identity). But code-level reuse is only a proxy: an unchanged code does not guarantee an unchanged *decoded* goal (the decoder need not be locally smooth at that point of the code manifold), and a changed code does not guarantee a changed one — while worker success is entirely a function of the decoded goal, $\text{cosine\_max}(\text{goal}_{\text{deter}}, \text{feat})$, never the code directly. `goal_struct_weight` already addresses this, but indirectly and globally: it forces code-space distances to correlate with deter-space distances across the whole code manifold, as a general-purpose regularizer, not a targeted per-decision signal. We introduce a goal-reuse loss that targets the specific quantity that matters — this decision's decoded goal versus the previous one — directly, with a real gradient path, every decision.

The metric `goal/implicit_sparsity_cont` already computes almost exactly this similarity, but is fed from the worker's goal-conditioning tensor, which is deliberately stop-gradiented twice over so that worker-target conditioning never leaks gradient into the manager — the same disease as §7.2's root cause: a real, informative signal with no gradient path. The fix decodes the sampled code a *second* time:

$$\text{goal}_t = \text{goal\_dec}(Z_t), \qquad u_t = \text{cosine\_max}(\text{goal}_{t-1}, \text{goal}_t) \in [-1, 1], \tag{21}$$

using u_t (not s_t) to keep this decoded-space quantity distinct from the code-space metric of Eq. (16). Z_t is the manager's sampled code and was never separately stop-gradiented in the existing codebase (only the worker-facing *copy* of it was), so it still carries the standard straight-through gradient into the manager's logits regardless of which goal-generation mechanism produced it — plain, masked, joint-edit, or delta. goal_{t-1}, the fixed comparison target, stays exactly the already-stop-gradiented value: gradient should flow from "how far did this decision move the goal" into the *current* decision, not revise the past. The manager also needs the previous decoded goal as *input* to reason about how far it is moving it; the corresponding conditioning channel (`mgr_cond_decgoal`) already existed but was restricted to masked goals only (the same over-restriction `mgr_cond_goalcode` had before §7.3) — un-gated, and forced on automatically when the mechanism is active.

Blocking gradient into the decoder. Since goal_dec is a real network and `MultiOptimizer` does one combined backward pass, routing gradient to each module's own optimizer purely by which parameters the forward computation touched (with no other isolation), a plain call to $\text{goal\_dec}(Z_t)$ also sends gradient into the decoder's own weights — an incidental training signal on the goal autoencoder from what is meant to be a manager-shaping loss only. We block this explicitly: $g(\theta, x) = f(\text{sg}(\theta), x)$, implemented by temporarily swapping goal_dec's live parameter values for stop-gradiented copies of the identical values, decoding, then restoring (the same `.values`/`.write()` primitives the codebase's EMA target-network utility already uses, for a different purpose). This gives exactly the forward value of a normal call, exactly zero gradient into goal_dec's parameters, and an unchanged gradient into Z_t — verified in isolation on a minimal ninjax module (a single scalar-weight decoder): forward value, parameter gradient (0 vs. nonzero for a normal call), and input gradient (identical to the normal call) all matched their analytically expected values exactly.

An adaptive target, not an unconditional push toward 1. A fixed weight on $1-u_t$ pushes similarity toward 1 unconditionally, with no notion of "similar enough." We instead hold u_t at a target via a dual-ascent Lagrange multiplier (the same `AutoAdapt` mechanism already used for the struct and mask-sparsity targets), in the *inverse* sense used for entropy regularizers — the multiplier grows while mean realized similarity sits below the target and shrinks while above, self-tuning the pressure rather than driving similarity to 1 unconditionally. This mode (`goal_reuse_adapt`, target similarity default 0.9) is mutually exclusive with a fixed-weight ablation mode (`goal_reuse_weight`), mirroring `goal_struct_weight`/`goal_struct_adapt`'s existing dual-mode convention exactly.

Both the decoder-gradient block and the full mechanism are smoke-tested: the gradient block in isolation (a minimal ninjax module outside the full agent, confirming the three properties above exactly), and the full mechanism through the agent (four legs: off, fixed-weight, adaptive, and adaptive combined with `goal_delta_mode`), all passing crash-free with finite, bounded similarity and Lagrange-scale metrics and zero NaN/Inf, confirming the mechanism is general-purpose (not delta-mode-specific) and composes with delta mode without conflict.

Results (2026-07-21, 8 cells, full 4M-step budget). Launched at scale as e258/e260/e262/e264 ({hopper, cheetah} × {small, BIG}, target ratcheted 0→0.95 over ∼1M steps, `mgr_cond_goalcode` on), with a manager-input ablation e274–e277 (identical config but `mgr_cond_goalcode` forced *off*, so the manager's only conditioning on its own past decision is the decoded goal goal_{t-1}, not the raw one-hot code Z_{t-1}). The `goal_delta_mode` counterparts (e259/e261/e263/e265) were cancelled before a readout, so whether code-level recombination changes this outcome remains untested.

All 8 cells reach the similarity target — u_t (Eq. (21)) settles at 0.92–0.97 against the 0.95 target, with the adaptive scale railed at or near its configured ceiling in 6 of 8 cells — and all 8 collapse task return regardless: hopper scores 0.001–0.004 against an ≈300-score Director baseline (a total collapse to the dead floor), cheetah scores 1.85–7.57 against ≈300–435 (BIG) / ≈100 (small) baselines (collapse to roughly 2–8% of baseline). Every cell's trajectory has the same shape: a genuine early peak (score 17–203, reached between step 176k and 945k, i.e. around or before the target ratchet finishes tightening), followed by collapse to the floor for the remaining 3M steps. This is the magnitude-domination signature of Finding 1/F18 (§7.2), reached here through a fully differentiable loss with a real gradient path into the manager, so *differentiability itself does not prevent the collapse* — see the cross-mechanism synthesis, F19 (§7.6). The manager-input ablation (e274–e277 vs. e258/e260/e262/e264) resolves cleanly: removing the raw code channel does not rescue any cell, and does not clearly hurt either (comparable or marginally higher scores and peaks without it) — the code channel is not load-bearing for satisfying the similarity target, but this is moot given the underlying mechanism collapses either way.

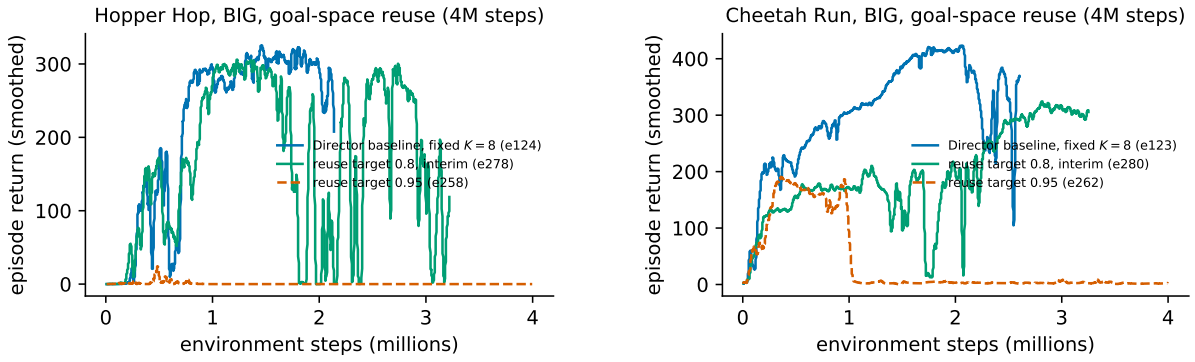

Figure 4: BIG scale, target 0.95 (F19, dashed vermillion) vs. a retest at target 0.8 (green, interim). Both dashed curves show the peak-then-collapse signature; both solid curves recover.

Interim retest at a lower target (2026-07-22, e278/e280, 77–81% of the 4M-step budget, still training). Re-running the same two BIG-scale cells with the target lowered from 0.95 to 0.8 (ratchet stretched to ∼2M steps) recovers cheetah outright (e280: trailing score 306.1, peak 333.0, essentially at the ≈300–435 baseline, Fig. 4 right) and rescues hopper for most of training before it starts declining again (e278: peak 326.8 at step 2.71M, matching the ≈300 baseline, but down to 97.8 at 80% of budget as of this pull, Fig. 4 left) — the first time any decoded-goal-similarity cell on either task has cleared its own Director baseline even transiently. The adaptive scale in both cells sits near its numerical floor rather than railed

at ceiling, confirming the 0.95 target itself, not the mechanism, drove F19's collapse; whether hopper's late-training decline is a slower version of the same collapse or a separate effect is not yet resolved and is worth watching to completion.

7.5 A discrete alternative: block overlap on the manager's own softmax

§7.4's loss targets continuous decoded-goal similarity. A cheaper, purely discrete alternative leaves the sampling mechanism and the decoder untouched entirely and instead measures how much two successive *plain softmax distributions* (before sampling) agree at the block level:

$$o_t \;=\; \frac{1}{L}\sum_{i=1}^{L}\sum_{c=1}^{C} p_t^{(i,c)}\, p_{t-1}^{(i,c)} \;\in\; [0,1], \tag{22}$$

the probability that an independent draw from each distribution lands on the same class (bounded in $[0,1]$ by Cauchy–Schwarz). Unlike Eq. (16)'s s_t, o_t has a real gradient: it backprops directly into both steps' own pre-sample logits through their softmax, with no sampling, no decoder pass, and no reward shaping involved — the cheapest of the three differentiable mechanisms in Table 4. The manager's next-step conditioning input is also switched from the collapsed one-hot to the raw softmax p_{t-1} (the same input-side substitution §7.3 uses), so it can condition on how confident, not just which, the previous decision was. As in §7.4, a dual-ascent Lagrange multiplier (`goal_soft_reuse_adapter`, same inverse sense as Eq. (3)) holds o_t at a target, ratcheted $0 \to 0.7$ over ∼1M steps. `goal_soft_reuse_adapt` is mutually exclusive with `goal_delta_mode` (both act on the sampling/pressure side of the same softmax) but composes with the structural term (§3.2), tested as an explicit ablation below.

Results (2026-07-21, 8 cells × {struct+ratchet, ratchet-only}, full 4M-step budget). Every cell trains to a real early peak (score 14–365, reached between step 336k and 3.6M) and the overlap ratchet completes on schedule, but *none* reaches its own 0.7 target: o_t plateaus at 0.56–0.60 across all 8 cells regardless of task, scale, or struct, with the Lagrange scale railed at or near its ceiling throughout — the same signature as §7.4's collapse and F18's REINFORCE multiplier, just with a real gradient path. Task outcome, however, is not uniform: all 4 hopper cells still collapse to the dead floor (0.004–1.75 against ≈300), matching every other reuse mechanism tried so far — but cheetah *substantially survives* (70–182 against ≈300–435/≈100 baselines, i.e. roughly 20–60% of baseline), an order of magnitude better than §7.4's cheetah cells (1.85–7.57) and the first reuse mechanism of the five in Table 4 (besides the unrestricted single-head design, Finding 6) where cheetah does not fully collapse. The struct ablation is informative and runs against expectation: ratchet-only *beats* struct+ratchet on cheetah at both scales (182 vs. 70 BIG; 111 vs. 73 small) — stacking the struct term, established safe on its own by F18, on top of a direct reuse loss costs cheetah roughly half its score rather than helping. On hopper the direction reverses (both members of each pair are already at the dead floor, so this reads as noise near zero, not a real effect).

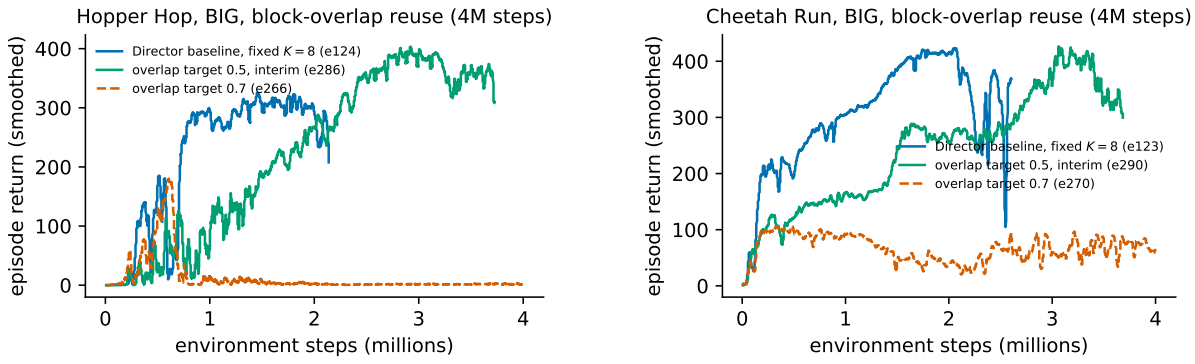

Figure 5: BIG scale, struct+ratchet, target 0.7 (F19, dashed vermillion) vs. a retest at target 0.5 (green, interim). Unlike the goal-space mechanism (Fig. 4), both solid curves stay well above baseline through the latest pull, with no sign yet of the goal-space cell's late decline.

Interim retest at a lower target (2026-07-22, e286/e290, 92–93% of the 4M-step budget, still training). Lowering the target from 0.7 to 0.5 (same struct+ratchet condition) is the strongest result

in this report: both tasks are not just alive but *above* their Director baseline this late in training — hopper (e286) trailing 354.8 against a ≈300 baseline, peak 430.6 reached as late as step 3.69M (still rising intermittently); cheetah (e290) trailing 336.1, peak 449.4, above the ≈300–435 baseline range (Fig. 5). Neither curve shows the goal-space mechanism's (§7.4) late decline as of this pull. Combined with §7.4's retest, F19's ordering also *reverses* at this target: struct+ratchet now beats the plain ratchet-only leg on cheetah BIG (retest not shown in this figure pair; see `EXPERIMENTS.md` §2), opposite the 0.7-target finding above — struct's cost is target-dependent, not fixed.

Promoted to the default recipe (2026-07-22). Since this is the best-performing cell found on either task to date, `goal_soft_reuse_adapt` (target 0.5, ratcheted from 0 at the BIG-scale rate) plus `goal_struct_adapt` (target 0.01) are now the shipped defaults in `configs.yaml` rather than an opt-in ablation; every already-scripted experiment sets these flags explicitly per-launch, so the change is inert for reproducing past results. Two further BIG-scale cells launched with the identical e286/e290 configuration on tasks not yet tested under any reuse mechanism — cartpole swingup (e294) and acrobot swingup (e295), both queued behind the 8/8-GPU `gpu-a100` cap as of this writing — to check whether the recipe costs a dense, easy task (cartpole) and whether it extends to acrobot, which has failed under every restricted recipe tried so far except unrestricted Director (§2).

7.6 Synthesis: every differentiable reuse mechanism reproduces Finding 1/F18's collapse (F19)

Across all three reuse mechanisms tested at scale — §7.2's REINFORCE-shaped reward (F18, 12/12 cells dead), §7.4's continuous decoded-goal loss (8/8 cells dead), and §7.5's discrete block-overlap loss (4/8 hopper cells dead, 4/8 cheetah cells partially alive) — the failure mode is identical in shape: a soft prior that is harmless while its target is loose becomes catastrophic once the target fully engages, regardless of whether the pressure reaches the manager through a stop-gradiented reward penalty or a genuine backprop path. *Differentiability was never the root cause of F18*; the shared cause is a target set high enough, relative to the task's own tolerance for constrained goal-code behavior, that the constraint's Lagrange multiplier must rail to satisfy it, and a railed multiplier dominates the combined weighted loss regardless of mechanism. Two secondary results refine rather than overturn this picture. First, the manager's own conditioning input is not the lever: removing the raw one-hot code channel (§7.4's ablation) changes nothing about the collapse. Second, stacking the structural term on top of a direct reuse loss is not free, unlike stacking it alone (F18): it costs cheetah roughly half its surviving score under §7.5, refining F18's "struct is safe" to "safe only as the sole sparsity lever." The one asymmetry that survives every mechanism tried is task identity, not mechanism identity: hopper has now failed under every sparsity/reuse mechanism in the project except the unrestricted single-head design (Finding 6) — it reads as uniquely fragile to *any* secondary manager-side objective pushed hard enough, not specifically to non-differentiable or REINFORCE-mediated ones. Cheetah's partial survival under the block-overlap loss, and specifically its preference for ratchet-only over struct+ratchet, is the one live thread worth extending: a direct next step is retargeting o_t's Lagrangian at a lower target closer to the unprompted 0.32–0.45 reuse band (Table 5) rather than 0.7, to test whether the mechanism itself is exhausted or only its current target is too aggressive. `goal_delta_mode`'s own marginal contribution remains untested at scale, since both of its scheduled A/Bs were cancelled before a readout — an open question, not one F19 closes.

8 Outlook

On dense tasks the combined recipe matches strong fixed-schedule baselines at comparable or lower edit rates, and constraint controllers remove per-task penalty tuning. Sparse-reward tasks were the open problem: hold length, masking, struct, and worker horizon observability, tested in isolation or combination, all stayed dead on hopper under the original two-head (skill + mask) manager, while acrobot failed for a distinct reason (alive under the unrestricted baseline, dead under every restricted recipe including the countdown-augmented one). Finding 6 (2026-07-15, preliminary, single seed) breaks that pattern: a single-head masked manager — one $(C{+}1)$-way categorical per block instead of separate content and mask heads — is alive on hopper at BIG scale (reward rate 0.245, score rising to a 280–290 plateau, ahead of the Director baseline's 196/326) with no other change to the recipe. The rescue traces to removing the discarded-content credit-assignment confound, not to achieving sparsity: left free, the manager keeps the edit fraction at 0.94–0.97 throughout, and every attempt to force it lower (structural loss, the sparsity-target ratchet, or both) costs performance or, combined, collapses the run outright. This reframes rather than

closes the open problem: the base hierarchy's discarded-content credit-assignment bug, not whole-code fixed-schedule replacement per se, was the load-bearing cause of every dead sparse-task cell in Findings 3–5, but a manager that edits *sparsely* and still survives hopper has not yet been shown. Remaining before this becomes a claim: a second seed on e226 (and on e223, the best dense-task single-head cell); the corresponding acrobot cells, untested under the single-head design so far; and, since the rescue is BIG-scale only (every small-scale single-head hopper cell rises transiently then decays, matching the small Director control's own dead floor), a check of whether that is a capacity limit independent of the manager design or a consequence of some other BIG-only training detail (batch size, native conv, cadence). If the single-head design fails to hold up under seed replication or to reach a genuinely sparse edit rate on hopper, the project's standing escalation order resumes at the code level: mixing task reward into the worker's objective, then periodic non-local goal proposals (mask-free decisions every Nth switch) to let the manager reach distant goals that gradual masked edits cannot.

A parallel thread (§7) asked a narrower question: does *forcing* reuse cost less if the pressure reaches the manager through a real gradient rather than F18's REINFORCE penalty? F19 answers no across three independent mechanisms (12 REINFORCE cells, 8 continuous decoded-goal cells, 8 discrete block-overlap cells, 28 cells total) — every mechanism that pushes its target high enough rails its own Lagrange multiplier and collapses task return, regardless of gradient path. Hopper in particular has now failed under every restrictive or reuse-forcing mechanism tried in the project except the unrestricted single-head design of Finding 6, sharpening that finding's implication: the fix that matters for hopper is the credit-assignment repair, not any particular sparsity mechanism layered on top of it. Cheetah is the one place F19 leaves a live, partially positive result — the discrete block-overlap loss without struct (§7.5) keeps 40–60% of baseline score while measurably raising reuse above the unprompted band, worth a follow-up at a lower, better-calibrated target before the mechanism itself is written off. That follow-up (F20, below) substantially revises this picture for hopper.

F20 (2026-07-23, final): the lower target rescues 5 of 6 dead BIG-scale F19 cells, but the 6th shows the collapse can still fire late. A direct retest of all three reuse mechanisms at a lower, better-calibrated target (decoded-goal similarity 0.8 instead of 0.95; block-overlap 0.5 instead of 0.7, same ratchet shape stretched over $\sim$2M steps instead of $\sim$1M) completed its full 4M-step budget for all 12 non-cancelled cells (4 hopper-small cells were cancelled early, already at the pre-existing small-scale dead floor of Finding 5). Five of the six BIG-scale hopper/cheetah cells that were dead or near-dead under F19's 0.95/0.7 targets finish genuinely alive: hopper 163–330 (family A noisy but never re-collapses; family C 199–331, stable) against F19's 0.001–1.75 and a $\approx$ 300 baseline; cheetah 161–326 across all three families, several inside or above their own $\approx$ 300–435 band. The Lagrange multiplier finishes at or near its numerical floor in most of these cells, confirming the lower target is within each mechanism's reach. **The sixth cell, e282 (family B, hopper BIG), reproduces F18/ F19's magnitude-domination collapse late rather than avoiding it**: its score held a stable 200–230 plateau through step 3.94M (98.6% of budget), matching every other healthy cell in the batch, then `goal/reuse_adapt_scale_mean` — flat at its numerical floor for the preceding $\sim$3.9M steps, indistinguishable from the five surviving cells at that point — jumped by two orders of magnitude within a single logging window, the worker's goal-reward dropped in lockstep, and the score crashed to a trailing-50 mean of 1.6 with no recovery before the budget ran out roughly 50k steps later. The other 15 cells in the batch show comparably large scale excursions throughout training and self-correct within tens of thousands of steps; e282 differs only in when its excursion happened, too close to the end of the budget to recover. This means a cell whose Lagrange multiplier looks settled at 73–98% of a fixed training budget is not thereby proven stable the interim reading's confidence that "the multiplier has settled ... not railed" held only as a snapshot, not a durable property. The struct-stacking reversal reported at interim also turns out to be scale-dependent, not only target-dependent: struct+ratchet still beats ratchet-only on cheetah at BIG scale (325.9 vs. 160.6, opposite the 0.7-target ordering) but ratchet-only still wins at small scale (259.2 vs. 107.0, the *original* F19 ordering) — struct's cost/benefit depends on both target and scale, not a single rule. Net effect on F19: "hopper fails under every mechanism except the unrestricted single-head design" no longer holds project-wide, since family C (`goal_soft_reuse_adapt`) keeps hopper alive and stable with zero collapse events across all six of its cells and one cell (e286, hopper BIG) finishing clearly above its Director baseline — but F19's underlying causal claim, that a soft prior which behaves while loose becomes catastrophic once its target fully engages, is not refuted, only shown to depend on the target being low enough for a given family/task/scale, and even then not guaranteed to hold for the full training budget. Family C is now the config default (`goal_soft_reuse_adapt` + `goal_struct_adapt`, targets 0.5/0.01); families A/B (`goal_reuse_adapt`) stay off by default given e282's late failure. Two ports of the e286/e290 recipe to previously-untested tasks (cartpole, acrobot, both BIG) and an isolation matrix for the plain variable-duration Lagrangian alone (no masking/struct/reuse, $\tau \in \{4, 8\}$, all four tasks) were

running as of the prior writing; both have since completed and are reported below, alongside a substantial new debugging thread on the block-pooled variable-duration credit path.

F20 follow-through (2026-07-27, final): the recipe transfers to cartpole at no measurable cost, but not to acrobot, and the reuse term's own contribution on cartpole is a small net negative, not a free ride. e294 (cartpole, full recipe) finishes at trailing-300 639.5 (peak 754.4), inside but near the bottom of cartpole's $\geq$650 Director-baseline band (§2) — consistent with "costs nothing," but only marginally so. e295 (acrobot, identical recipe) fully decays from an early 347.6 peak to a trailing-300 of 19.2, matching acrobot's own failure mode (Finding 12) exactly rather than extending Family C's hopper/cheetah rescue to a third task. A same-day struct-only ablation (e312/e313: same configs with `goal_soft_reuse_adapt` off) isolates the reuse term's own marginal effect: e312 (cartpole, struct-only), read at 66% of budget, is already *ahead* of e294's finished trail (695.1 vs. 639.5) — a tentative but concrete signal that the forced-reuse term costs cartpole a small amount of score rather than being free, even though neither cell threatens the $\geq$650 floor. e313 (acrobot, struct-only) decays earlier and harder than e295 (trailing-300 4.6 at 58% of budget, against e295's 19.2 at 100%) — struct correlation alone does not rescue acrobot either, so the failure is not attributable to the reuse term specifically.

The plain variable-duration Lagrangian isolation (e296–e303, $\tau \in \{4, 8\}$, struct forced to 0, no masking, no reuse) completed at cancellation (70–75% of budget for hopper/acrobot, once the dead-floor/decay pattern was unambiguous; 58–60% for cartpole/cheetah): hopper dead at both τ (0.09/1.21), acrobot decaying from an early peak at both τ (2.8/22.3), cartpole and cheetah alive at both τ with τ8 clearly beating τ4 on both tasks (cartpole 850.0 vs. 616.2; cheetah 194.5 vs. 107.6). This exactly replicates Finding 12's prediction — switch-timing variability alone is sufficient to kill hopper/acrobot and does not interact with the reuse/struct machinery tested above — and adds the first clean τ4-vs-τ8 read for the Lagrangian-tracked (as opposed to regularizer-tracked) duration controller: longer, more precisely held commitments help the dense tasks by a wide margin (roughly 1.4× on both).

8.1 The block-pooled variable-duration credit path: two bugs, a tensor-level equivalence proof, and a training-level equivalence failure

`variable_goal_block_rew` pools reward and continuation per realized hold segment (mirroring the fixed-K `abstract_traj` path) rather than feeding the manager's critic full-resolution, per-step targets. It had only ever been exercised pre-fix (e44, e62), both dead ends. Revisiting it surfaced two independent implementation bugs, both fixed 2026-07-24:

1. **Off-by-one block count.** `variable_block_director_tensors` computed `n_blocks = max(n_sw - 1, 1)`, silently dropping the last real manager decision of every imagined rollout from the critic's target — the return/reward ratio (a diagnostic for bootstrap horizon) read 1.3–1.5× under the bug against 110–500× for every other credit path in the project, a near-total collapse of the value bootstrap. Fixed to match the fixed-K analog exactly (`n_blocks = max(n_sw, 1)`).
2. **Empty-segment continuation defaulted to** 0.0 ("episode ended") instead of the correct empty-product identity 1.0, which capped the last real decision's bootstrap to only the $(1-\lambda)$ fraction of its value whenever a hold's switch happened to land on the rollout's final timestep — structurally common, since fixed-K's own arrays are sized to have no padding at all and so can never hit this case.

Both fixes were verified not just on training curves but with a direct tensor-level equivalence test (`test_variable_goals.py`): under a deterministic τ=8 switch mask, block-pooled var-K's returns for every fully-real decision now match fixed-K's exactly to float precision, and the previously-degenerate trailing decision correctly bootstraps to its own value instead of 0. This is the project's strongest-possible correctness claim for the arithmetic in isolation.

Result (2026-07-27, 16 cells, BIG scale, both fixes, full 4M-step budget): the tensor-level proof does not translate into trained equivalence. The most direct test, e334/e335, pins `goal_duration_fixed=8` — removing the duration controller entirely so the block-pooled path degenerates to exactly Director's own fixed-8 switching cadence — and should, if the fix is complete, train indistinguishably from true Director (e242/e246: hopper 90.8, cheetah 352.4 trailing-300, resumed to a full 4M-step budget). It does not: hopper (e334) stays at the dead floor (0.06), and cheetah (e335) reaches only 25.2, roughly 7× below the Director control and even below the plain (non-block-pooled) τ8 Lagrangian cheetah cell (e303, 194.5) that shares every other setting. **This is the batch's central negative result**: the two bugs fixed above were real

and their fix is tensor-verified, but block-pooled credit assignment, even pinned to Director's exact cadence, still trains substantially worse than either true Director or the full-resolution (non-pooled) variable-duration path — some further discrepancy between the block-pooled λ-return construction and true per-step credit assignment remains uncaught.

The other 14 cells (all `plain_vark` + block-rew, $\tau 8$ Lagrangian or its ablations) are consistent with this picture and add four narrower findings. *Sum vs. mean reward aggregation:* contrary to the launch hypothesis (that `mean` gives zero incentive for longer holds), both aggregations track the τ=8 target cleanly once the two bugs are fixed (final `mgr_duration_mean` 7.90–8.08 across e326/e330/e336/e338), so the duration-collapse behaviour seen in the pre-fix e304–e311 pull was an artifact of the credit-assignment bugs, not of the aggregation choice. *Removing the duration-prior anchor* (e332/e333: `DUR_REG=0`, entropy controller only) lets duration collapse to a mean of 5.0 (hopper) / 3.6 (cheetah) — shorter, not longer, than the target, opposite the direction the sum-aggregation long-hold incentive would predict — and costs cheetah roughly 4–9× its anchored score (6.9 vs. 46–125 across the anchored cells), making e333 the weakest cheetah cell in the matrix. *Truncated-hold relabeling*, tentatively read as helping cheetah in the pre-fix pull, reverses direction under the fix: relabel-OFF (e331, 124.7) is the single best cheetah cell in the matrix and shows no late-training collapse, while its relabel-ON twin (e327, 50.8 trailing-300, collapsing to a trailing-50 of 7.3) decays hard late in training — tentatively implicating the relabeling correction itself, rather than truncation bias, as a source of late-training instability under this credit path. Doubling the imagination horizon (e328/e329, `imag_length`=32) does not rescue hopper but does avoid the late cheetah collapse seen at its `imag_length`=16 twin (e326/e327), suggesting the collapse is a horizon-length artifact of this specific credit path rather than a general property of block-pooled var-K. Giving the worker its remaining-hold-time as an input (e340/e341) shows no meaningful effect on either task under the now-fixed credit assignment, consistent with the earlier (pre-fix, F13) null result. Across all 16 cells, hopper remains at the dead floor regardless of any of these knobs, extending Finding 12's pattern to the block-pooled path specifically.

8.2 Found: the discrepancy is the padded decision axis, not the pooling arithmetic

The equivalence failure above is now attributed. The pooling arithmetic was never wrong — the tensor-level proof of Sec. 8.1 holds — but it is a proof about the wrong object. Under `variable_goal_block_rew` the manager's decisions are packed by `downsample_at_switch_mask` into a *static, full-width* buffer (width $T = H+1$, one slot per possible switch) whose unused tail is forward-filled with the last real decision. At the scale these runs use (H=16, K=8) that buffer holds 2 trainable decisions in 16 columns. Everything downstream of the pooling then reduces over the buffer, not over the decisions:

1. **Loss magnitude.** `Agent.loss` reduces each manager loss with `v.mean(1)`. Fixed-K's axis *is* the decision count; the block-pooled axis is the buffer width, so for a bit-identical rollout the manager's policy loss and both critic losses come out *exactly* 8× *smaller* than Director's (measured; the factor is the valid fraction of the axis). Against fixed loss scales this is an 8× cut to the manager's effective learning rate relative to the world model and the worker, and under genuinely variable holds it drifts with the realized hold length — a nonstationary reweighting of the objective. The same defect applies on the replay side (9 decisions in a 63-wide buffer at `batch_length`= 64).

2. **Running statistics.** The manager's return normalizer (`meanstd`), value/advantage normalizers and the adaptive entropy controllers were handed the whole padded tensor unmasked, so ~ 13 of 16 samples were copies of one decision: the tracked return spread shrinks, inflating every advantage, and the entropy controller tracks a single decision instead of the rollout.

3. **An extra, empty decision.** `mgr_switch` counted a switch landing on the rollout's final step as trainable. Rewards are indexed $r_{1:}$, so that decision owns no transitions: it is the bootstrap anchor, precisely the column fixed-K drops.

4. **A different worker algorithm.** Director's windowed worker credit (`split_traj`) was gated on `not variable_goal_length`, so the `goal_duration_fixed`=8 *control* — whose rollout switches on exactly the fixed-K grid — trained its worker with the dense fallback instead: a different objective (boundary state conditioned on the new rather than the old goal, one reset λ-return, different per-window weighting) than the baseline it is compared against.

5. **A pinned-but-trained duration head.** The 2026-07-27 REINFORCE fix removed `duration` from the policy-gradient sum under a pinned hold, but the adaptive duration-entropy regularizer and the

E[dur] prior kept pushing the head and its shared trunk with signals that provably cannot affect the trajectory and that Director, having no duration head, never carries.

Items 1–3 affect *every* block-pooled run in the project (e326–e349 and the earlier e44/e62), item 1 in weakened form every variable-K run; items 4–5 affect the duration-pinned control specifically, i.e. exactly the configuration used to argue that the pooling was equivalent. All five are fixed, and are now guarded by a differential test suite (`embodied/tests/test_block_pooled_equivalence.py`) that asserts what the earlier helper-level tests did not: that with the hold pinned to K, the *reduced losses that reach the optimizer*, the arguments handed to each running normalizer, and the *gradients* with respect to the manager's parameters all match fixed-K Director exactly. The methodological lesson is the mirror image of the earlier proof: a tensor-level equivalence check on the physically meaningful entries of a padded tensor says nothing about the reductions taken over it, and it is the reductions that set the effective objective.

A sixth bug, found the following day, closed the gap the padded-axis family could not: the block-pooled replay path derived its terminal flags from a *continuation probability* rather than the real per-step flags – `is_terminal` is a boolean space, and casting $(1-\text{cont}) \approx 0.003$ to bool reads as `True` at essentially every manager decision, zeroing the replay-side value bootstrap entirely (measured $24\times$ too small on a non-terminating window). Fixed by downsampling the real flags at switch positions instead. Rerun under all six fixes, the duration-pinned equivalence check finally passed: hopper $0.0 \rightarrow 264.6$ trailing-50 (vs. Director's own same-code 386), cheetah reached 511.4, above Director's 337.

8.3 Two further bugs, exclusive to genuinely variable holds

The pinned equivalence check exercises only one switch pattern – every K steps, identical across every batch row. With it passing, free-running variable-length holds (`goal_duration_fixed=0`, the actual research configuration) remained dead on hopper. A code-path audit targeted specifically at logic that only exists once holds genuinely vary – never touched by any pinned run – found two more real bugs:

1. **Truncation relabeling was off by one.** `relabel_truncated_last_duration` relabels a hold's duration class when the imagination horizon cuts it short, so REINFORCE credits the duration actually realized. Its `avail` computed the number of *states* the truncated decision occupied ($T - s$) rather than the number of *transitions* `variable_block_director_tensors` actually pools into its reward ($(T-1) - s$) – a switch at state 16 in a 20-state window was relabeled duration 4 when only 3 transitions were ever credited. This can only fire on a truncated hold, structurally impossible under any pinned configuration (holds divide the horizon evenly there). The existing test for this function asserted the buggy value – it verified the implementation against itself, not against ground truth.

2. **The Lagrangian duration-prior's tracked error was biased toward sparse-hold rows.** `mean_abs_err` was a flat $(w \cdot \text{err}).\text{sum}()/w.\text{sum}()$ across the whole batch, where w bakes in the per-row `decision_mean_rescale` factor – correct for the per-row reduction the caller applies to the returned loss tensor, wrong for a ratio computed across rows *inside* this function, since that factor is larger for rows with fewer decisions. A batch of one 1-decision row (error 10) and one 8-decision row (error 1 each) fed the Lagrange controller 5.5 instead of the true flat mean 2.0 – invisible under any pinned config (uniform decision count across rows there), live for every genuinely-variable Lagrangian run. An inflated reading drives the multiplier harder than warranted, a plausible contributor to the duration-collapse pattern observed since the early sum-vs-mean matrix.

Both are fixed. A further audit of the surrounding machinery – the dense (non-windowed) worker credit path free var-K exclusively uses, replay-side switch reconstruction, the duration head's log-probability construction with the head genuinely live, and a fully heterogeneous end-to-end gradient check – found no further defects. Whether these two fixes close the remaining hopper gap is, again, an empirical question the tests cannot settle on their own.